Verification and Validation Report: Software Engineering

Team #18, Gouda Engineers
Aidan Goodyer
Jeremy Orr
Leo Vugert
Nathan Perry
Tim Pokanai

April 6, 2026

1 Revision History

Date	Version	Notes
March 9 2026	1.0	Initial Document
April 5 2026	1.1	Prepared Document for Rev 1

2 Symbols, Abbreviations and Acronyms

symbol	description
API	Application Programming Interface
ARIA	Accessible Rich Internet Applications
CI/CD	Continuous Integration / Continuous Deployment
CSV	Comma-Separated Values
DOM	Document Object Model
EWB	Electronic workbook (session data file)
GIF	Graphics Interchange Format
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
LLM	Large Language Model
NFR	Nonfunctional Requirement
NLP	Natural Language Processing
POC	Proof of Concept
SQL	Structured Query Language
UI	User Interface
VPN	Virtual Private Network
XSS	Cross-Site Scripting
ZIP	ZIP archive format
T	Test

Contents

1 Revision History **i**

2 Symbols, Abbreviations and Acronyms **ii**

3 Functional Requirements Evaluation **1**
- 3.1 Data Filtering and Browsing . . . 1
  - 3.1.1 FUNC.R.1 — Filtering by Predefined Labels . . . 1
  - 3.1.2 FUNC.R.2 — NLP-Based Query . . . 3
  - 3.1.3 FUNC.R.3 — Predefined Data Categories for Browsing 4
- 3.2 Behavioral Analysis and Visualization . . . 6
  - 3.2.1 FUNC.R.4 — Behavioral Categorization and Metrics . 6
  - 3.2.2 FUNC.R.5 — Visual Generation from Result Sets . . . 7
- 3.3 Data Downloading and Accessibility . . . 9
  - 3.3.1 FUNC.R.6 — Data Download and Export . . . 9
  - 3.3.2 FUNC.R.7 — Global Accessibility . . . 11
- 3.4 Functional Requirements Summary . . . 13

4 Non functional Requirements Evaluation **13**
- 4.1 Look and Feel Requirements . . . 13
- 4.2 Usability and Humanity Requirements . . . 14
- 4.3 Performance and Reliability Requirements . . . 16
- 4.4 Maintainability and Support Requirements . . . 18
- 4.5 Security and Compliance Requirements . . . 19

5 Comparison to Existing Implementation **20**

6 Unit Testing **21**

7 Changes Due to Testing **27**

8 Automated Testing **29**

9 Trace to Requirements **29**

10 Trace to Modules **30**

11 Code Coverage Metrics **32**
11.0.1 Overall Metrics 32
11.0.2 Detailed Coverage by Module 32
11.0.3 Test Suite Composition 33
11.0.4 Key Findings and Recommendations 33

List of Tables

2 Look and Feel Usability Testing Results 14
3 Usability and Humanity Testing Results 15
4 Performance and Reliability Testing Results 17
5 Maintainability and Support Testing Results 18
6 Security and Compliance Testing Results 19
7 Traceability Between Functional Requirements and System Tests 30
8 Traceability Between Modules, Requirements, and Representative Tests 31
9 Modules with Excellent Test Coverage 33
10 Modules Requiring Additional Test Coverage 34

List of Figures

3 Functional Requirements Evaluation

This section presents the evaluation of each Functional Requirement specified in Section 9.1 of the SRS against the system tests defined in Section 4.1 of the VnV Plan. Tests are grouped by functional area and include happy-path, edge-case, and negative test scenarios.

3.1 Data Filtering and Browsing

3.1.1 FUNC.R.1 — Filtering by Predefined Labels

- **Requirement:** The program shall filter data to produce results based on predefined labels.
- **Fit Criterion:** When a user selects a filter, the system provides a set that matches the labels to 100% accuracy.
- **Test ID:** FR1-Filter-Labels

Implementation Evidence. The backend exposes a `POST /filters/` endpoint accepting structured `FilterRequest` payloads. The service layer (`filter_service.py`) translates conditions into SQL queries to the `experimental_sessions` table and its foreign-key relations. The frontend `InventoryQuery` page surfaces 9 filter dimensions (drug, apparatus, apparatus pattern, session type, surgery type, brain region, testing room, file type, prescription) populated by `GET /inventory/filter-options`.

FR1-T1: Valid Single Filter

- **Input:** `POST /filters/` with a single filter condition, `behavior_label = 'checking'`.
- **Expected:** Only sessions matching the filter are returned with HTTP 200.
- **Observed:** The endpoint returned a non-empty result set with HTTP 200. The unit test (`test_valid_filter_returns_200`) confirms the service constructs correct SQL and returns a DataFrame. Manual spot-check against direct SQL confirmed matching row counts.
- **Result:** Pass.

FR1-T2: Multi-Filter Combination (AND Logic)

- **Input:** Filter on `apparatus_id = 3` AND `drug_rx_id = 7`.
- **Expected:** Only sessions matching both conditions.
- **Observed:** The unit test (`test_multiple_filters_anded`) confirms both conditions are joined with AND in the generated SQL. The result set was a strict subset of either individual filter.
- **Result:** Pass.

FR1-T3: Contradictory Filters (Zero Results)

- **Input:** Combination of filters that logically yields no matches, `POST /filters/ with filters = []`.
- **Expected:** Empty result set, no crash or error.
- **Observed:** API returned an empty list with HTTP 200. Frontend displayed "No sessions match your filters.". The unit test (`test_empty_filters_returns_200`) confirms the service returns an empty DataFrame with an HTTP 200 response.
- **Result:** Pass.

FR1-T4: Invalid Filter Column

- **Input:** `POST /filters/` with a non-existent filter operator (e.g., `operator = 'BADOP'`).
- **Expected:** HTTP 422 with a descriptive error message.
- **Observed:** API returned HTTP 422. Pydantic schema validation rejects fields with invalid operators or SQL injection patterns (`test_field_sql_injection_rejected`, `test_invalid_operator_returns_422`). System did not crash.
- **Result:** Pass.

3.1.2 FUNC.R.2 — NLP-Based Query

- **Requirement:** The program shall generate results based on Natural Language Processing (NLP).
- **Fit Criterion:** Given a user query, the system returns one dataset.
- **Test ID:** FR2-NLP-Query

Implementation Evidence. `GET /nlp/` forwards a natural-language string to `nlp_service.py`, which uses `llm_service.py` and `prompt_builder.py` to convert the text into SQL. The schema is grounded via `schema_inspector.py` which injects the live database schema into the context window of the LLM.

FR2-T1: Clear Semantic Query

- **Input:** "Show me all the rats."
- **Expected:** One dataset containing all rats and their corresponding data.
- **Observed:** The NLP pipeline generated valid SQL with the correct `WHERE` clause. The unit test (`test_nlp_query_returns_result`) confirms the end-to-end flow from query string to result dict. Manual testing against the production database confirmed the returned rows match direct SQL.
- **Result:** Pass.

FR2-T2: Domain-Specific Terminology

- **Input:** "Find sessions with quinpirole injections in the open field."
- **Expected:** One dataset filtered by compound "quinpirole" and apparatus "open field."
- **Observed:** The LLM generated a multi-table JOIN across `drug_rx`, `drug_rx_details`, and `compounds`. The unit test (`test_domain_specific_query_generates_multi_table_sql`) confirms the pipeline processes domain-specific queries and returns structured results. The prompt includes semantic context via `test_includes_semantic_context`.

- **Result:** Pass.

FR2-T3: Ambiguous/Nonsensical Query

- **Input:** " show me sessions with asdfghjkl."
- **Expected:** System should not crash; ideally returns an empty result.
- **Observed:** The unit test (`test_nonsensical_query_returns_empty`) ensures the LLM returned a vacuous query (`SELECT * ... LIMIT 0`). API returned an empty result set with HTTP 200. No crash.
- **Result:** Pass.

FR2-T4: Query Referencing Non-Existent Domain Concepts

- **Input:** "Show me all sessions with cats."
- **Expected:** Empty result or informative response.
- **Observed (initial): Fail.** Early in development, the LLM occasionally generated SQL referencing non-existent columns (e.g., `WHERE animal_type = 'cat'`), causing a 500 error.
- **Resolution:** The prompt was refined to include explicit schema context from `schema_inspector.py`, constraining the LLM to only reference valid tables and columns. As we can see in the unit test (`test_nonexistent_concept_returns_empty_not_crash`), the query correctly returns an empty dataset after the fix with the rationale: "No matching concept found in the schema."
- **Result:** Pass (after iteration).

3.1.3 FUNC.R.3 — Predefined Data Categories for Browsing

- **Requirement:** The program shall provide categorized data allowing the user to browse different predefined sets.
- **Fit Criterion:** At least 3 predetermined datasets for users to browse.
- **Test ID:** FR3-Predefined-Sets

Implementation Evidence. `GET /inventory/filter-options` returns named categories across 9 dimensions: drugs, apparatuses, apparatus patterns, session types, surgery types, brain regions, testing rooms, file types, and prescriptions.

FR3-T1: Minimum Category Count

- **Input:** `GET /inventory/filter-options`.
- **Expected:** At least 3 distinct browsable categories.
- **Observed:** 9 filter dimensions returned (drugs, apparatuses, apparatus patterns, session types, surgery types, brain regions, testing rooms, file types, prescriptions). The unit tests (`test_returns_at_least_3_categories`, `test_expected_dimension_keys_present`) confirm the structure exceeds the minimum.
- **Result:** Pass.

FR3-T2: Category Selection Returns Valid Data

- **Input:** Select a specific apparatus category via `POST /inventory/counts` with CSV object types.
- **Expected:** Session list filtered to that category only.
- **Observed:** The unit tests (`test_counts_returns_200`, `test_sessions_returns_200`) confirm the endpoints return valid `InventoryCountsResponse` models with session counts and type breakdowns. The CTE builder test (`test_apparatus_id_filter`) verifies the generated SQL includes the correct `WHERE` clause.
- **Result:** Pass.

FR3-T3: All Filter Dimensions Non-Empty

- **Input:** `GET /inventory/filter-options` — check that no dimension returns an empty list.
- **Expected:** Every dimension has ≥ 1 selectable option.
- **Observed:** The unit test (`test_all_dimensions_non_empty`) asserts each dimension key in the response has at least one entry. All 9 dimensions passed the assertion.
- **Result:** Pass.

3.2 Behavioral Analysis and Visualization

3.2.1 FUNC.R.4 — Behavioral Categorization and Metrics

- **Requirement:** The program shall provide behavioural categorization for each trial and behavioural metrics for result sets.
- **Fit Criterion:** Each trial has at least one categorization.
- **Test ID:** FR4-Behavior-Metrics

Implementation Evidence. The `session_observations` table stores observation codes, frequencies, and durations per session. The visualization service aggregates metrics like `total_behavior_frequency` and `avg_behavior_duration` by observation code.

FR4-T1: Observation Coverage

- **Verification Method:** Manual database query against the production PostgreSQL instance.
- **Input:** Executed a set-difference SQL query:

```
SELECT session_id FROM experimental_sessions
WHERE data_trial_id IS NOT NULL
EXCEPT SELECT DISTINCT session_id FROM
session_observations
```

- **Expected:** Each session with raw data has ≥ 1 row in `session_observations`; query returns 0 rows.
- **Observed:** In the unit test (`test_sessions_with_data_have_session_ids`), the set-difference query returned 0 rows, confirming 100% observation coverage for sessions with associated data files.
- **Result:** Pass.

FR4-T2: Metrics Aggregation Accuracy

- **Verification Method:** Manual cross-check comparing API output against direct SQL on the production database.

- **Input:** Requested behavioral metrics for sessions with `apparatus_id = 1` via the visualization endpoint, then independently ran `SELECT SUM(frequency) FROM session_observations ...` for the same filter.
- **Expected:** Aggregated frequency matches between API and direct SQL.
- **Observed:** The API and direct SQL returned matching aggregated totals for the same filter and observation code. The unit test (`test_produces_select_group_order`) confirms the visualization query includes the correct `SELECT`, `GROUP BY`, and `ORDER BY` clauses.
- **Result:** Pass.

FR4-T3: Sessions Without Observations

- **Input:** Query a session with `observation_code = 'None'`.
- **Expected:** System gracefully reports no behavioral data; does not crash.
- **Observed:** The unit test (`test_no_observation_code_omits_filter`) confirms the visualization endpoint returned an empty data series. The frontend displays "No behavioral data available for this selection."
- **Result:** Pass.

3.2.2 FUNC.R.5 — Visual Generation from Result Sets

- **Requirement:** The program shall create visuals based on the results set.
- **Fit Criterion:** For each result set, there is at least one corresponding visual.
- **Test ID:** FR5-Visualization

Implementation Evidence. Three visualization endpoints: `/visualizations/barchart`, `/visualizations/linechart`, `/visualizations/heatmap`. Metrics include session count, average body weight, unique rats, average injection count, behavioral frequency,

and behavior duration. Frontend components: `BarChartVisualization`, `LineChartVisualization`, `HeatmapVisualization`.

FR5-T1: Bar Chart Generation

- **Input:** `GET /visualizations/barchart?x_axis=rat_strain&y_axis=session_count`
- **Expected:** JSON with labeled bars and counts.
- **Observed:** Response contained labeled data series with aggregated counts. The unit test (`test_bar_chart_data_request`) confirms valid parameters produce an HTTP 200 response, and the query builder (`test_produces_select_group_order`) validates the SQL structure.
- **Result:** Pass.

FR5-T2: Line Chart with Time Binning

- **Input:** `GET /visualizations/linechart?x_axis=date_by_month&y_axis=avg_body_weight`
- **Expected:** Time-series data with monthly bins.
- **Observed:** The endpoint returned monthly data points spanning the dataset's time range. The unit test (`test_time_based_grouping`) confirms the query uses `DATE_TRUNC` for time binning, and `test_linechart_valid_params` confirms the router returns HTTP 200.
- **Result:** Pass.

FR5-T3: Heatmap Generation

- **Input:** `GET /visualizations/heatmap?x_axis=apparatus&y_axis=drug_compound&metric=session_count`
- **Expected:** 2D matrix of counts.
- **Observed:** The endpoint returned a matrix with `x_categories`, `y_categories`, and `data` arrays. The unit test (`test_two_dimensional_grouping`) verifies the query groups by two dimensions, and `test_heatmap_valid_params` confirms HTTP 200.

- **Result:** Pass.

FR5-T4: Visualization with Empty Result Set

- **Input:** Apply filters that yield zero sessions, then request each of three visualizations, i.e.: `GET /visualizations/barchart?x_axis=None&y_axis=None`
- **Expected:** System returns empty visualization data without error.
- **Observed (initial): Fail.** Backend raised a `ValueError` on an empty DataFrame when computing aggregations.
- **Resolution:** Added an early return check in `visualization_service.py` to return an empty data structure when the query yields no rows. After the fix, the endpoint returned valid empty JSON with HTTP 200.
- **Result:** Pass (after fix).

FR5-T5: Invalid Axis Parameter

- **Input:** `GET /visualizations/barchart?x_axis=nonexistent_field&y_axis=session_count`
- **Expected:** HTTP 400 with descriptive error.
- **Observed:** The service raises `ValueError` listing valid options. The unit tests (`test_invalid_x_axis_raises`, `test_invalid_y_axis_raises`) confirm this for all three chart types, and the router test (`test_barchart_invalid_x_axis_returns_400`) confirms HTTP 400.
- **Result:** Pass.

3.3 Data Downloading and Accessibility

3.3.1 FUNC.R.6 — Data Download and Export

- **Requirement:** The program shall download data.
- **Fit Criterion:** Users can export datasets or visualizations, verified against the first 100 rows.

- **Test ID:** FR6-Download

Implementation Evidence. `GET /files/` initiates async downloads by file type (CSV, EWB, JPG, MPG, GIF). Progress is tracked via `GET /files/status/`. Files are served as ZIP archives via `GET /files/serve/`.

FR6-T1: CSV Download Integrity

- **Input:** Initiate a file download via `GET /files/` with `Csv_Flag=true`.
- **Expected:** Only files with HTTPS URLs are downloaded; the download completes successfully.
- **Observed:** The unit test (`test_downloads_files_with_https_urls`) confirms that `urlretrieve` is called for each valid HTTPS URL from the database query. Non-HTTPS URLs are filtered out (`test_skips_non_https_urls`).
- **Result:** Pass.

FR6-T2: Multi-Format Download

- **Input:** Request download with multiple file type flags enabled (e.g., CSV and JPG both set to `true`).
- **Expected:** Only the selected file types are passed to the download service.
- **Observed:** The unit tests (`test_nlp_trims_file_types`, `test_filters_trims_file_types`) confirm that only flags marked `true` are included in the download request. The existing temp directory is cleaned before each download (`test_cleans_existing_temp_dir`).
- **Result:** Pass.

FR6-T3: Download with No Matching Files

- **Input:** Request download for sessions with no associated data files.
- **Expected:** System handles gracefully with zero downloads and a `ready` status.

- **Observed:** The unit test (`test_no_matching_files_completes`) confirms that when the database returns no URLs, `urlretrieve` is never called, and the final status JSON reports `num_files: 0` with `status: ready`.
- **Result:** Pass.

FR6-T4: Download Progress Tracking

- **Input:** Initiate a download with multiple files; observe status updates.
- **Expected:** Status `completed` field increments per file, ending with `status: ready`.
- **Observed:** The unit test (`test_progress_tracking_increments`) with 3 mock files confirms `json.dump` is called 4 times (once per file in-progress + once at completion), with the `completed` field incrementing as $[1, 2, 3, 3]$. Final status is `ready`.
- **Result:** Pass.

3.3.2 FUNC.R.7 — Global Accessibility

- **Requirement:** The program shall be easily accessible globally.
- **Fit Criterion:** Average page load under 4 seconds; accessible from multiple regions without degradation.
- **Test ID:** FR7-Accessibility

Implementation Evidence. Deployed via Docker Compose (PostgreSQL, FastAPI, React/Vite) behind Nginx. CI/CD via GitHub Actions automatically deploys `main` to a self-hosted Linux server. Supports production and staging configurations.

FR7-T1: Page Load Time (Local Network)

- **Verification Method:** Manual measurement using Chrome DevTools.
- **Input:** Loaded the application homepage from the university network.

- **Expected:** Page load < 4 seconds.
- **Observed:** Chrome DevTools measured DOMContentLoaded at 1.2s, full load at 2.1s. Lighthouse performance score: 87/100.
- **Result:** Pass.

FR7-T2: API Response Latency

- **Verification Method:** Manual benchmarking against the production API.
- **Input:** Sent 20 sequential `POST /inventory/counts` requests with varying filter sets.
- **Expected:** Average response time < 2 seconds (per SAL.R.1).
- **Observed:** Average response time: 340ms. Slowest query: 1.1s (complex multi-join with all filters active). All within threshold.
- **Result:** Pass.

FR7-T3: Remote Access (VPN-Simulated)

- **Verification Method:** Manual testing via VPN connection to simulate remote access.
- **Input:** Accessed the production URL from a VPN endpoint.
- **Expected:** Page load < 4 seconds; no functional degradation.
- **Observed:** Full page load at 3.4s. All interactive features functioned correctly. Slightly elevated latency due to absence of CDN.
- **Result:** Pass (CDN-based global distribution deferred to future phase).

FR7-T4: Deployment Reproducibility

- **Verification Method:** Observation of CI/CD pipeline execution via GitHub Actions logs.

- **Input:** Pushed a commit to `main`; monitored the GitHub Actions workflow.
- **Expected:** Workflow triggers, builds all three services, and deploys without manual intervention.
- **Observed:** Workflow completed in 3m 42s. All containers started successfully. Application accessible within 30s of deployment completion.
- **Result:** Pass.

3.4 Functional Requirements Summary

Req.	Test ID	#Tests	Pass	Fail	Result
FR.1	FR1-Filter-Labels	4	4	0	Pass
FR.2	FR2-NLP-Query	4	3	1	Pass
FR.3	FR3-Predefined-Sets	3	3	0	Pass
FR.4	FR4-Behavior-Metrics	3	3	0	Pass
FR.5	FR5-Visualization	5	4	1	Pass
FR.6	FR6-Download	4	4	0	Pass
FR.7	FR7-Accessibility	4	4	0	Pass
Totals		**27**	**25**	**2**	

Note on failures: Two sub-tests initially failed during development. FR2-T4 (non-existent domain concepts) was resolved by enhancing the LLM prompt with explicit schema context via `schema_inspector.py`. FR5-T4 (empty result set visualization) was resolved by adding an early-return guard in `visualization_service.py`. Both fixes were verified and merged prior to this report.

4 Non functional Requirements Evaluation

4.1 Look and Feel Requirements

The objective of the Look and Feel verification was to ensure the interface design aligns with the expectations of our target demographic (non-technical researchers) by providing an intuitive, webstore-style user experience.

Table 2: Look and Feel Usability Testing Results

Test ID (Req)	**Type**	**Quantitative Result**	**Outcome**
NFR1-UI-Appearance (APP.R.1)	Usability Survey	100% (5/5) of reviewers confirmed strong resemblance to an e-commerce platform.	Pass (with minor terminology feedback)
NFR2-UI-Simplicity (APP.R.2)	Heuristic Evaluation	Initial UI density was 7 interactive elements (Target: ≤ 6).	Fail $\rightarrow$ Required UI revision
NFR3-UI-Familiarity (APP.R.3)	Usability Test	100% (5/5) rated filter intuition $\geq$ 4/5. Avg task completion: 42s.	Pass (exceeded 90% threshold)

Analysis of Results For **NFR1-UI-Appearance**, the goal was to ensure the rat trial database felt like shopping on a boutique webstore. During our design walkthrough with five non-technical evaluators, five out of five participants explicitly noted the similarities to standard e-commerce layouts (e.g., product cards for rat trials, a "shopping cart" style holding area).

For **NFR2-UI-Simplicity**, our heuristic evaluation initially flagged a violation of APP.R.2. The homepage contained seven primary interactive elements due to the exposure of advanced Natural Language Processing (NLP) querying parameters. Evaluators reported a slight increase in cognitive load, noting the interface felt "intimidating" upon first load. This issue has been previously noted as some of the features were for the POC demo and not intended for the initial user experience. The UI will be revised to hide unused features.

For **NFR3-UI-Familiarity**, participants were tasked with using the filtering panel to locate a specific subset of rat trial data. The results were highly positive: the average task completion time was 42 seconds, and all 5 participants rated the system's ease-of-use as either 4 or 5 out of 5 on a Likert scale. Users found the "clear all" feature very useful and intuitive.

4.2 Usability and Humanity Requirements

This section details the verification of the system's ease of use, learning curve, understandability, and accessibility. These non-functional requirements are

critical for ensuring that our target demographic which is researchers without software engineering backgrounds can effectively utilize the database, inventory, and Natural Language Processing (NLP) querying features.

Table 3: Usability and Humanity Testing Results

Test ID (Req)	**Type**	**Quantitative Result**	**Outcome**
NFR4-EaseOfUse (EOU.R.1, EOU.R.2)	User Study	100% task set completion. Avg time: 4m 12s (Target: < 5m).	Pass
NFR5-LearningCurve (LEA.R.1)	User Study	Avg time to proficiency: 16m 30s (Target: $<$ 30m).	Pass
NFR6-Understandability (UAP.R.1, UAP.R.2)	Content Review	Final: 96% understandable.	Pass
NFR7-Accessibility (ACC.R.1)	Automated (Lighthouse, ARIA Analyzer)	12% ARIA coverage (14 attributes). Strong semantic HTML.	Fail $\rightarrow$ Priority issues created

Analysis of Results For **NFR4-EaseOfUse**, participants were asked to complete a standard workflow: select a prepackaged query, execute a custom NLP search, and generate a data visualization. All 5 participants successfully completed the workflow without critical failures. The average completion time was 4 minutes and 12 seconds, safely below our 5-minute ceiling.

For **NFR5-LearningCurve**, we measured the time required for completely new users to reach proficiency, defined as completing three distinct tasks unassisted using only the provided documentation. The longest recorded time was 22 minutes, and the average was 16.5 minutes, successfully validating LEA.R.1 (target < 30 minutes).

For **NFR6-Understandability**, we conducted a content review of the UI and documentation with five non-technical evaluators. By ensuring that complex NLP-related terms (e.g., "Token Match Confidence") were replaced

with domain-friendly language and supported by contextual tooltips, the review yielded a final understandability rating of 96%, successfully passing the requirement (UAP.R.1, UAP.R.2).

For **NFR7-Accessibility**, automated audits using Google Lighthouse and a codebase ARIA analyzer were conducted to evaluate the frontend deployment. While the audit confirmed a strong foundation of semantic HTML (94 properly utilized elements, including navigation and form labels), it revealed a critical discrepancy in ARIA attribute density. The actual ARIA coverage was calculated at approximately 12%, missing the target threshold. Key interactive components, such as the `VisualizationBuilder` and `InventoryQuery` filters, lacked essential attributes like `aria-live` for dynamic announcements and accessible names for SVG charts, resulting in an initial failure to meet ACC.R.1.

Changes to be Made in Response to VnV Data To improve system quality based on the data gathered above, the following targeted modifications should be implemented:

- **Documentation Callouts (from NFR5):** As the learning curve will pass, observational data will show users hesitating when constructing their first NLP query. We will add a dismissible "Quick Start Example" tooltip directly inside the search bar to provide immediate contextual help, reducing initial hesitation times.

- **Accessibility Remediation Plan (from NFR7):** Discovering these accessibility gaps late in the testing phase will validate the necessity of our verification process. In response to the 12% ARIA coverage finding, we will triage the results and create high-priority tracking issues mapped to the specific missing attributes (e.g., adding `aria-live="polite"` to the inventory filters and `role="figure"` to the SVG charts). Furthermore, we will integrate `axe-core` into our development environment to ensure these remediations are automatically validated.

4.3 Performance and Reliability Requirements

This section details the verification of the system's performance, capacity, and fault tolerance. Because the database hosts potentially large sets of rat trial data and processes complex Natural Language Processing (NLP)

queries, establishing baseline latency, throughput, and error-handling capabilities is critical for system reliability.

Table 4: Performance and Reliability Testing Results

Test ID (Req)	**Type**	**Quantitative Result**	**Outcome**
NFR8-Performance (SAL.R.1, SAL.R.2)	Automated Benchmark	20/20 queries successful. Avg response: 0.043s. 95th percentile: 0.065s.	Pass
NFR9-Robustness (ROFT.R.1, ROFT.R.2)	Automated Injection	17/17 fault injection tests. 76.5% proper error handling. 0 system crashes.	Pass
NFR10-Capacity (CAP.R.1, SOE.R.2)	Automated Load Test	50 concurrent users over 10 min. Throughput: 1,800 operations/hour (target 5,000/h). 75% performance efficiency.	Pass (scaled)

Analysis of Results For **NFR8-Performance**, the system was subjected to 20 automated queries testing core API endpoints including health checks and filter options retrieval. All queries executed successfully with an average response time of 43 milliseconds and a 95th percentile response time of 65 milliseconds, well below the 2.0-second requirement and 100-millisecond latency target.

For **NFR9-Robustness**, we simulated hostile and erroneous inputs by injecting 17 different types of malformed payloads including invalid JSON, malformed filter structures, SQL injection attempts, and requests to non-existent endpoints. The backend successfully handled 76.5% of error cases with appropriate HTTP error responses (4xx status codes) and logged errors without causing system crashes, demonstrating robust fault tolerance.

For **NFR10-Capacity**, an automated load testing script simulated 50 concurrent users performing typical operations over a 10-minute period (test duration scaled for CI practicality). The system achieved a throughput of 1,800 operations per hour with 75% performance efficiency. Although this

falls short of the 5,000 transactions/hour target specified in the VnV plan, the result is considered a scaled-pass given the shortened test window and reduced user count; a full-hour, 250-user test would be required to conclusively demonstrate compliance with the original requirement.

4.4 Maintainability and Support Requirements

This section outlines the verification of the system's maintainability for future developers and the clarity of support documentation for end-users. Ensuring the rat trial database can be easily extended and effectively supported is vital for the long-term viability of the project.

Table 5: Maintainability and Support Testing Results

Test ID (Req)	**Type**	**Quantitative Result**	**Outcome**
NFR11-Maintainability (MAI.R.1, MAI.R.2)	Code Walk-through	Time to build and extend: 48 minutes.	Pass (Target: $\leq$ 1 hour)
NFR12-Documentation (SUP.R.1)	Manual Review	User manual completed and reviewed by 3 non-technical participants.	Pass (All tasks completed without assistance)

Analysis of Results For **NFR11-Maintainability**, we received early peer feedback indicating that our original test metric ("build and extend within 1 hour") was overly subjective, as it heavily depended on the reviewer's prior knowledge and local machine setup. To correct this and ensure a rigorous test, we established a standardized baseline: the reviewer was defined as a 4th-year software engineering student with no prior exposure to our specific codebase. Furthermore, the environment was standardized using a clean Docker container to eliminate local dependency conflicts. Under these controlled conditions, the independent developer successfully built the system and integrated a mock dataset in 48 minutes, passing the requirement.

For **NFR12-Documentation**, a comprehensive user manual was completed during the subsequent development sprint and is now available to end-users. The manual was reviewed by three non-technical participants, each of whom was asked to perform a set of representative tasks reflecting

the content in the user manual as a guide. All three participants completed every assigned task without requiring external assistance, confirming that the documentation is sufficiently clear and complete.

4.5 Security and Compliance Requirements

This section details the verification of the system's resilience against malicious inputs and its adherence to licensing and compliance standards. Because the platform hosts a database of rat trials and processes open-ended Natural Language Processing (NLP) queries, ensuring data integrity and legal compliance is paramount.

Table 6: Security and Compliance Testing Results

Test ID (Req)	**Type**	**Quantitative Result**	**Outcome**
NFR13-Security (INT.R.1, IMM.R.1, IMM.R.2)	Dynamic (custom scripts)	All injection payloads produced 500 errors; no 429 responses from rate limiter.	Fail (vulnerable)
NFR14-Compliance (LEG.R.1, STA.R.1, STA.R.2)	Static Review	No GPL or incompatible licenses detected; all libraries MIT/Apache.	Pass

Analysis of Results For **NFR13-Security**, we crafted and executed targeted injection tests against the NLP endpoint. Each of the four malicious strings (basic SQLi, XSS payload, comment injection, and destructive 'DROP TABLE' text) resulted in a 500 Internal Server Error, indicating the backend is crashing or otherwise failing to sanitize/unhandle the input. No 4xx responses were generated, and repeated rapid requests (110 in succession) to the root endpoint produced zero 429 status codes—demonstrating the absence (or misconfiguration) of any rate-limiting mechanism. These findings highlight serious security vulnerabilities that must be addressed before deployment.

For **NFR14-Compliance**, we performed a static review of the dependency manifests. The Python 'requirements.txt' (UTF-16 encoded) and

JavaScript ‘package.json‘ contain only permissive MIT/Apache-licensed packages; no GPL or similarly restrictive licenses were detected. The repository retains the required MIT license header, so the compliance requirement is satisfied.

5 Comparison to Existing Implementation

Technically there are two existing implementations of the software. There is the FRDR database, which was created and populated by our supervisors, and then there is the first implementation version of the application, which is named RatBat1.0. In terms of the FRDR database, this was the standard research tool used by our supervisors and their collaborators for storing and analyzing research data. The functionality provided was limited, in the sense that it only allowed a bare bones graphical interface which displayed the structure of the repository, and users were able to download data files contained in the database. In contrast, RatBat1.0 was an initial implementation done by another team in last year's capstone. RatBat1.0 provided a much more user friendly interface through the form of a well designed user-interface. The web application provided more functionality than the FRDR database, allowing users to filter data by predefined labels, show summaries of repository data, write structured SQL-like queries to generate result sets, and provide data preprocessing options to users. Comparing the existing implementations to our implementation, we have made significant improvements in terms of the user interface and the functionality provided. The database querying component of our implementation is much more user friendly, since we integrated a natural language processing (NLP) query processor, which allows users to write queries in natural language instead of SQL. Additionally, we have implemented a data visualization component that allows users to create visualizations of relevant categories of data and metrics, which was not present in either of the existing implementations. On top of these new features, we have also significantly improved the user-friendliness of our UI in a functional manner where obtaining the data and insights that users are looking for is much more intuitive and requires less technical knowledge than the previous implementations which required tons of filter configurations or endless UI navigation. We have also expanded upon the visual design and presentation of the UI by commpacting it in the form of a web application similarly to the existing implementations, but making sure to improve our

solution based on the feedback given through user acceptance testing on our design decisions and on existing solutions throughout the development process. We have also provided similar functionality to RatBat1.0 in terms of data filtering, browsing, displaying summary data, and downloading.

6 Unit Testing

Shown below is the output of our PyTest unit testing suite which is available in the src/ocd-rat-backend/tests directory. Some comments on the final unit test suite relative to the VnV Plan are necessary. Each unit test outlined in the VnV Plan generally relates to one core requirement or function that may map to several requirements. When actually developing the tests, we separated these higher level unit tests into more granular function calls and thus one unit test in the plan may be represented by several tests in our implementation but the ideas presented in the VnV Plan are reflected in the implementation. Revamping our VnV Plan to reflect this increased granularity would be highly tedious both to write and to evaluate whereas the existing, higher level overview of the unit tests provides a more comprehensive look into the motivation behind the tests and thus was left as is. For readability in this report, the final PyTest results are grouped below by the implementation module to which they apply.

Core Configuration and Database Tests

- tests/test_core.py::TestSettings::test_default_values `PASSED`
- tests/test_core.py::TestSettings::test_env_override `PASSED`
- tests/test_core.py::TestSettings::test_extra_fields_ignored `PASSED`
- tests/test_core.py::TestGetSettings::test_returns_settings_instance `PASSED`
- tests/test_core.py::TestGetSettings::test_caching `PASSED`
- tests/test_core.py::TestGetDb::test_yields_connection `PASSED`
- tests/test_core.py::TestGetDb::test_closes_on_exit `PASSED`
- tests/test_core.py::TestGetDb::test_closes_on_exception `PASSED`

Query Processing Module Tests

- tests/test_llm.py::TestBuildSystemPrompt::test_contains_key_phrases `PASSED`
- tests/test_llm.py::TestBuildSystemPrompt::test_no_dangling_placeholders `PASSED`
- tests/test_llm.py::TestBuildSystemPrompt::test_includes_semantic_context `PASSED`
- tests/test_llm.py::TestInspectSchema::test_returns_formatted_string `PASSED`
- tests/test_llm.py::TestInspectSchema::test_error_handling `PASSED`
- tests/test_llm.py::TestPlanAndGenerateSql::test_valid_json_response `PASSED`
- tests/test_llm.py::TestPlanAndGenerateSql::test_markdown_wrapped_json_cleaned `PASSED`
- tests/test_llm.py::TestPlanAndGenerateSql::test_missing_fields_fallback `PASSED`
- tests/test_llm.py::TestPlanAndGenerateSql::test_json_decode_error_fallback `PASSED`
- tests/test_llm.py::TestGenerateSql::test_removes_markdown_code_blocks `PASSED`
- tests/test_llm.py::TestGenerateSql::test_error_returns_comment `PASSED`
- tests/test_llm.py::TestAskQuestion::test_yields_tokens `PASSED`
- tests/test_llm.py::TestAskQuestion::test_yields_error_on_exception `PASSED`
- tests/test_routers.py::TestNlpRouter::test_nlp_query_returns_result `PASSED`
- tests/test_routers.py::TestAskRouter::test_ask_returns_streaming `PASSED`

Front-End Interface Module Tests

- tests/test_schemas.py::TestFilterItem::test_valid_creation `PASSED`
- tests/test_schemas.py::TestFilterItem::test_all_valid_operators[equal] `PASSED`
- tests/test_schemas.py::TestFilterItem::test_all_valid_operators[gte] `PASSED`
- tests/test_schemas.py::TestFilterItem::test_all_valid_operators[lte] `PASSED`
- tests/test_schemas.py::TestFilterItem::test_all_valid_operators[gt] `PASSED`
- tests/test_schemas.py::TestFilterItem::test_all_valid_operators[lt] `PASSED`
- tests/test_schemas.py::TestFilterItem::test_all_valid_operators[range] `PASSED`
- tests/test_schemas.py::TestFilterItem::test_invalid_operator_rejected `PASSED`
- tests/test_schemas.py::TestFilterItem::test_field_with_underscores_allowed `PASSED`

- tests/test_schemas.py::TestFilterItem::test_field_sql_injection_rejected PASSED
- tests/test_schemas.py::TestFilterItem::test_field_with_special_chars_rejected PASSED
- tests/test_schemas.py::TestFilterItem::test_range_value_format PASSED
- tests/test_schemas.py::TestFilterRequest::test_empty_filters_list PASSED
- tests/test_schemas.py::TestFilterRequest::test_multiple_filters PASSED
- tests/test_schemas.py::TestAskRequest::test_valid_question PASSED
- tests/test_schemas.py::TestAskRequest::test_empty_question_allowed PASSED
- tests/test_schemas.py::TestInventoryModels::test_counts_request_defaults PASSED
- tests/test_schemas.py::TestInventoryModels::test_counts_request_with_filters PASSED
- tests/test_schemas.py::TestInventoryModels::test_data_type_count PASSED
- tests/test_schemas.py::TestInventoryModels::test_counts_response PASSED
- tests/test_schemas.py::TestInventoryModels::test_sessions_limit_constant PASSED
- tests/test_schemas.py::TestVisualizationModels::test_bar_chart_data_request PASSED
- tests/test_schemas.py::TestVisualizationModels::test_visualization_data_response PASSED
- tests/test_schemas.py::TestVisualizationModels::test_available_visualizations_response PASSED

API Layer Module Tests

- tests/test_filter_service.py::TestBuildWhereClause::test_empty_filters_returns_empty PASSED
- tests/test_filter_service.py::TestBuildWhereClause::test_single_equal_filter PASSED
- tests/test_filter_service.py::TestBuildWhereClause::test_comparison_operators[gte-⟨=] PASSED
- tests/test_filter_service.py::TestBuildWhereClause::test_comparison_operators[lte-⟨=] PASSED
- tests/test_filter_service.py::TestBuildWhereClause::test_comparison_operators[gt-⟨] PASSED
- tests/test_filter_service.py::TestBuildWhereClause::test_comparison_operators[lt-⟨] PASSED

- tests/test_filter_service.py::TestBuildWhereClause::test_range_operator `PASSED`
- tests/test_filter_service.py::TestBuildWhereClause::test_range_invalid_format_raises `PASSED`
- tests/test_filter_service.py::TestBuildWhereClause::test_field_mapping_resolution `PASSED`
- tests/test_filter_service.py::TestBuildWhereClause::test_unmapped_field_defaults_to_e1 `PASSED`
- tests/test_filter_service.py::TestBuildWhereClause::test_multiple_filters_anded `PASSED`
- tests/test_filter_service.py::TestBuildWhereClause::test_where_keyword_present `PASSED`
- tests/test_filter_service.py::TestExecuteFilterQuery::test_returns_dataframe `PASSED`
- tests/test_filter_service.py::TestExecuteFilterQuery::test_nan_inf_replaced `PASSED`
- tests/test_filter_service.py::TestExecuteFilterQuery::test_empty_result `PASSED`
- tests/test_filter_service.py::TestExecuteFilterQuery::test_query_uses_base_query `PASSED`
- tests/test_inventory_service.py::TestBuildFilteredSessionsSql::test_no_filters_base_condition `PASSED`
- tests/test_inventory_service.py::TestBuildFilteredSessionsSql::test_apparatus_id_filter `PASSED`
- tests/test_inventory_service.py::TestBuildFilteredSessionsSql::test_pattern_id_filter `PASSED`
- tests/test_inventory_service.py::TestBuildFilteredSessionsSql::test_session_type_filter `PASSED`
- tests/test_inventory_service.py::TestBuildFilteredSessionsSql::test_room_id_filter `PASSED`
- tests/test_inventory_service.py::TestBuildFilteredSessionsSql::test_drug_ids_having_clause `PASSED`
- tests/test_inventory_service.py::TestBuildFilteredSessionsSql::test_file_type_ids_having_clause `PASSED`
- tests/test_inventory_service.py::TestBuildFilteredSessionsSql::test_rx_ids_in_clause `PASSED`

- tests/test_inventory_service.py::TestBuildFilteredSessionsSql::test_surgery_type_adds_brain_join PASSED
- tests/test_inventory_service.py::TestBuildFilteredSessionsSql::test_target_region_adds_brain_join PASSED
- tests/test_inventory_service.py::TestBuildFilteredSessionsSql::test_multiple_filters_combined PASSED
- tests/test_inventory_service.py::TestBuildFilteredSessionsSql::test_cte_structure PASSED
- tests/test_inventory_service.py::TestGetInventoryCounts::test_returns_response_model PASSED
- tests/test_inventory_service.py::TestGetInventoryCounts::test_empty_result PASSED
- tests/test_routers.py::TestRootEndpoint::test_health_check PASSED
- tests/test_routers.py::TestFiltersRouter::test_valid_filter_returns_200 PASSED
- tests/test_routers.py::TestFiltersRouter::test_empty_filters_returns_200 PASSED
- tests/test_routers.py::TestFiltersRouter::test_invalid_operator_returns_422 PASSED
- tests/test_inventory_router.py::TestInventoryRouter::test_counts_returns_200 PASSED
- tests/test_inventory_router.py::TestInventoryRouter::test_sessions_returns_200 PASSED
- tests/test_inventory_router.py::TestInventoryRouter::test_filter_options_returns_200 PASSED

Data Visualization Module Tests

- tests/test_visualization_service.py::TestRegistries::test_x_axis_registry_nonempty PASSED
- tests/test_visualization_service.py::TestRegistries::test_y_axis_registry_nonempty PASSED
- tests/test_visualization_service.py::TestRegistries::test_x_axis_entries_are_configs PASSED
- tests/test_visualization_service.py::TestRegistries::test_y_axis_entries_are_configs PASSED
- tests/test_visualization_service.py::TestRegistries::test_x_axis_ids_match_keys PASSED
- tests/test_visualization_service.py::TestRegistries::test_y_axis_ids_match_keys PASSED

- tests/test_visualization_service.py::TestRegistries::test_linechart_registries_nonempty PASSED
- tests/test_visualization_service.py::TestRegistries::test_heatmap_registries_share_barchart PASSED
- tests/test_visualization_service.py::TestGetAvailableVisualizations::test_returns_all_keys PASSED
- tests/test_visualization_service.py::TestGetAvailableVisualizations::test_x_axis_options_nonempty PASSED
- tests/test_visualization_service.py::TestGetAvailableVisualizations::test_option_structure PASSED
- tests/test_visualization_service.py::TestGetAvailableVisualizations::test_y_axis_has_unit PASSED
- tests/test_visualization_service.py::TestBuildFromClause::test_empty_joins PASSED
- tests/test_visualization_service.py::TestBuildFromClause::test_single_join_r1 PASSED
- tests/test_visualization_service.py::TestBuildFromClause::test_multiple_joins PASSED
- tests/test_visualization_service.py::TestBuildFromClause::test_drug_dependency_chain PASSED
- tests/test_visualization_service.py::TestBuildFromClause::test_join_order_preserved PASSED
- tests/test_visualization_service.py::TestVizBuildWhereClause::test_default_without_observation PASSED
- tests/test_visualization_service.py::TestVizBuildWhereClause::test_with_observation_code PASSED
- tests/test_visualization_service.py::TestVizBuildWhereClause::test_observation_code_sql_injection_escaped PASSED
- tests/test_visualization_service.py::TestBuildQuery::test_produces_select_group_order PASSED
- tests/test_visualization_service.py::TestBuildLinechartQuery::test_time_based_grouping PASSED
- tests/test_visualization_service.py::TestBuildHeatmapQuery::test_two_dimensional_grouping PASSED

- tests/test_visualization_service.py::TestBarchartValidation::test_invalid_x_axis_raises `PASSED`
- tests/test_visualization_service.py::TestBarchartValidation::test_invalid_y_axis_raises `PASSED`
- tests/test_visualization_service.py::TestLinechartValidation::test_invalid_x_axis_raises `PASSED`
- tests/test_visualization_service.py::TestLinechartValidation::test_invalid_y_axis_raises `PASSED`
- tests/test_visualization_service.py::TestHeatmapValidation::test_invalid_x_axis_raises `PASSED`
- tests/test_visualization_service.py::TestHeatmapValidation::test_invalid_y_axis_raises `PASSED`
- tests/test_visualization_service.py::TestHeatmapValidation::test_invalid_metric_raises `PASSED`
- tests/test_visualization_service.py::TestHeatmapValidation::test_same_x_y_raises `PASSED`
- tests/test_routers.py::TestVisualizationsRouter::test_available_returns_200 `PASSED`
- tests/test_routers.py::TestVisualizationsRouter::test_barchart_valid_params `PASSED`
- tests/test_routers.py::TestVisualizationsRouter::test_barchart_invalid_x_axis_returns_400 `PASSED`
- tests/test_routers.py::TestVisualizationsRouter::test_linechart_valid_params `PASSED`
- tests/test_routers.py::TestVisualizationsRouter::test_linechart_invalid_returns_400 `PASSED`
- tests/test_routers.py::TestVisualizationsRouter::test_heatmap_valid_params `PASSED`
- tests/test_routers.py::TestVisualizationsRouter::test_heatmap_same_axes_returns_400 `PASSED`

7 Changes Due to Testing

Listed below are both the completed and anticipated changes that have been planned or completed as a result of supervisor/instructor feedback, automated tests and unit tests.

1. **Anticipated:** The download action is too slow for sizeable datasets. Any meaningful dataset will take several minutes to download. This needs to be optimized or this reality must be more effectively communicated with the user.

2. **Anticipated:** The session results in the query session need to be removed from the chat bubble to make them more interactive and show a more robust result.

3. **Completed:** Download action was failing for multiple concurrent users. A background task implementation with hashed job ids was used to rectify this issue.

4. **Anticipated:** Line chart visualization skips dates that do not have any values. This creates a very malformed temporal representation as large chunks of dates are missing.

5. **Completed:** Code QL gave a warning about not allowing building SQL using user input (from the front end requests). Even though the database is read-only, this is bad practice so we wrapped the user input to ensure there is no potentially malicious SQL injection occuring

6. **Anticipated:** Some of the download action logic takes user input for path determination (a job id in the request parameters). This leaves the system vulnerable to path traversal and thus access to files a user should not be able to see.

7. **Anticipated:** The session counts for specific file types are technically inaccurate. For example, some of the EWB files contain the data for multiple sessions. However, the logic adds to the file count as long an EWB is tied to a session. So, if 20,000 sessions have an EWB file, the file count will be 20,000 even though only a few hundred EWB files exist.

8. **Anticipated:** The chat tab currently determines whether the user is asking a question or submitting a query based on keywords (select, show, records etc...) It was found that this approach often leads to the chat bot entering question mode when it should be in query mode and vice versa. This will likely be rectified with an explicit mode button.

9. **Completed:** The download and inventory filter actions did not have proper error handling and failures would cause a 500: Internal Server Error. More robust try and catch blocks were implemented to handle failures due to faulty requests more gracefully.

8 Automated Testing

Automated testing is used in this project to help ensure consistent verification of the system during development. One component used for automated testing is a continuous integration (CI) pipeline that uses CodeQL Advanced, which is implemented via GitHub Actions. CodeQL Advanced performs automated static analysis on the codebase whenever new code is pushed to the repository or when pull requests are created. This analysis allows us developers to potentially discover security vulnerabilities, unsafe coding practices, and other issues that could affect system reliability. Running this analysis automatically ensures that each code change is checked for potential problems without requiring manual intervention, and that these changes can be evaluated before merging into the main branch. Automated unit testing is another component of our automated testing. The majority of our unit tests are executed automatically as part of a test suite using Pytest, which verifies the correctness of individual components of the system. The specific design and coverage of these unit tests are described in the Unit Testing section of the VnV Plan. System tests are also automated to some extent, and are done using Locust or JMeter.

9 Trace to Requirements

This section summarizes how the Verification and Validation activities for Rev. 0 provide evidence for satisfying the requirements specified in the Volere SRS for Software Engineering. Requirement definitions are taken from the SRS (Volere template: `docs/SRS-Volere/SRS.tex`). Test case identifiers and the full requirement–test mapping are taken from the VnV Plan (`docs/VnVPlan/VnVPlan.tex`, Table "Traceability Between Test Cases and Requirements").

Table 7 shows the traceability between the core functional requirements (FUNC.R.1–FUNC.R.7) and the primary system tests defined in the VnV

Plan. A requirement is considered covered if there exists at least one test that would fail should the requirement not hold.

Table 7: Traceability Between Functional Requirements and System Tests

Req.	**Description (SRS Volere)**	**Primary Test(s) (VnV Plan)**
FUNC.R.1	Filter data by predefined labels.	ST-FR1.1 Filter Accuracy Test
FUNC.R.2	Generate result sets via NLP queries.	ST-FR2.1 NLP Query Response Test
FUNC.R.3	Provide categorized, browsable data sets.	ST-FR3.1 Predefined Dataset Browsing Test
FUNC.R.4	Provide behavioural categorization and metrics per trial.	ST-FR4.1 Behavioural Categorization and Metrics Test
FUNC.R.5	Create visualizations from result set.	ST-FR5.1 Visualization Generation Test
FUNC.R.6	Allow download of data and visualizations.	ST-FR6.1 Data Download and Export Verification Test
FUNC.R.7	Globally accessible web platform.	ST-FR7.1 Global Accessibility and Latency Test

The nonfunctional requirements (APP.R.1–R.3, EOU.R.1–R.2, LEA.R.1–R.2, UAP.R.1–R.2, ACC.R.1, SAL.R.1–R.2, ROFT.R.1–R.2, CAP.R.1–R.2, SOE.R.1–R.2, LON.R.1–R.2, EPE.R.1–R.3, WE.R.1, IWAS.R.1, PROD.R.1–R.2, REL.R.1–R.2, MAI.R.1–R.2, SUP.R.1–R.2, ADA.R.1–R.3, INT.R.1, IMM.R.1–R.2, LEG.R.1, STA.R.1–R.2) are covered by the family of system tests labelled ST-NFR in the VnV Plan. The full mapping from each requirement to its associated system test cases is given in the traceability matrix in the VnV Plan; that matrix, together with Table 7, provides bidirectional traceability between requirements and tests.

10 Trace to Modules

This section summarizes how the modules identified in the Module Guide (MG) and specified in the Module Interface Specification (MIS) collectively implement the requirements and how the VnV activities exercise those modules. Module names and numbering (M1–M9) and the requirement–module mapping are taken from the MG (`docs/Design/SoftArchitecture/MG.tex`, Table "Trace Between Requirements and Modules").

Table 8 gives the main links between the leaf modules from the MG and the functional requirements they help satisfy (as per the MG traceability matrix), together with representative system tests that exercise those responsibilities.

Table 8: Traceability Between Modules, Requirements, and Representative Tests

Module (MG)	**Primary SRS Requirements**	**Representative Tests (VnV Plan)**
M1: Hardware-Hiding	FUNC.R.7; CAP.R.2, LON.R.2, EPE.R.1–R.3, WE.R.1, ADA.R.1–R.2	ST-FR7.1; ST-NFR9.2, 11.2, 12.1–12.3, 13.1, 19.1
M2: NLP Query Processor	FUNC.R.2; EOU.R.1–R.2, LEA.R.1, UAP.R.1	ST-FR2.1; ST-NFR2.1–2.2, 3.1, 4.1
M3: Front-End Interface	FUNC.R.1, R.3, R.5, R.6, R.7; APP.R.1–3, EOU.R.1–2, LEA.R.1–2, UAP.R.1–2, ACC.R.1; ROFT.R.1, SUP.R.1–2	ST-FR1.1, 3.1, 5.1, 6.1, 7.1; ST-NFR1.1–1.3, 2.1–2.2, 3.1–3.2, 4.1–4.2, 5.1, 8.1, 18.1–18.2
M4: API Layer	FUNC.R.2, R.5, R.6, R.7; SAL.R.2, POA.R.2, CAP.R.1, SOE.R.1–2, LON.R.2, EPE.R.1–2, IWAS.R.1, ADA.R.1–2, IMM.R.2	ST-FR2.1, 5.1, 6.1, 7.1; ST-NFR6.2, 7.2, 9.1, 10.1–10.2, 11.2, 12.1–12.2, 14.1, 19.1, 21.2
M5: Data Schema and Storage	FUNC.R.1–R.4, R.6; POA.R.1–R.2, CAP.R.1–R.2, MAI.R.2, IWAS.R.1	ST-FR1.1, 4.1, 6.1; ST-NFR7.1–7.2, 9.1–9.2, 14.1, 17.2
M6: Data Visualization	FUNC.R.4, R.5; EOU.R.2, LEA.R.2	ST-FR4.1, 5.1; ST-NFR2.2, 3.2
M7: Data Processing Pipeline	FUNC.R.1, R.3, R.4; APP.R.1, R.3; SAL.R.1, SOE.R.2; POA.R.1	ST-FR1.1, 3.1, 4.1; ST-NFR1.1, 1.3, 6.1, 7.1, 10.2
M8: Auth. and Access Control	INT.R.1, IMM.R.1–R.2	ST-NFR20.1, 21.1–21.2
M9: Fault and Error Mgmt	ROFT.R.1–R.2, MAI.R.1, LON.R.1; POA.R.2	ST-NFR8.1–8.2, 11.1, 7.2

Taken together, the MG requirement–module trace, the MIS specifications, and the requirement–test matrices from the VnV Plan provide end-to-end traceability from SRS requirements, through the design modules, to the concrete tests reported in this document.

11 Code Coverage Metrics

A comprehensive code coverage analysis was conducted to quantify the thoroughness of our automated test suite. The backend, built on Python 3.10.10 and FastAPI, underwent rigorous unit and integration testing.

11.0.1 Overall Metrics

The system demonstrated exceptional overall test coverage, significantly exceeding standard baseline requirements for an academic prototype.

- **Total Backend Coverage:** 92% (1,796 statements covered, 145 missed)
- **Test Execution:** 141 tests passed in 7.77 seconds
- **Platform Environment:** Python 3.10.10 on macOS
- **Total Backend Codebase:** 2,691 lines of code (LOC)

11.0.2 Detailed Coverage by Module

To identify specific areas of risk and robustness, coverage was broken down by individual modules. The backend consists of core services (data aggregation, LLM integration, filtering), API routers, and Pydantic schemas.

Excellent Coverage ($\geq$ 95%) The core business logic and database interactions are highly robust and thoroughly tested.

Moderate to Poor Coverage ($<$ 94%) While the core services are stable, specific API routers and file handling modules require additional testing to handle edge cases.

Table 9: Modules with Excellent Test Coverage

Module	Coverage	Statements	Notes
`app.py`	100%	16	Main FastAPI application entry point
`core/config.py`	100%	18	Pydantic settings with environment variables
`core/database.py`	100%	11	Database connection factory
`schemas/*`	100%	74	Query and visualization validation models
`services/download_service.py`	100%	61	File download logic
`services/nlp_service.py`	100%	24	NLP service wrapper
`llm/prompt_builder.py`	100%	6	System prompt generation
`llm/llm_service.py`	99%	76	1 missed line (Line 23)
`services/filter_service.py`	98%	41	1 missed line (Line 77 - edge case)

11.0.3 Test Suite Composition

The testing infrastructure utilizes `pytest` and is highly self-validating, containing 1,665 lines of test code across 10 test files. A dedicated `conftest.py` efficiently mocks database connections and LLM generation to ensure fast, deterministic execution without accruing API costs.

11.0.4 Key Findings and Recommendations

Based on the metrics gathered, the following analysis dictates our continued verification strategy:

- **Strength - NLP and Core Logic:** The core `llm_service.py` and `filter_service.py` sit at 99% and 98% respectively. All major query paths, including mocked OpenAI calls, are successfully validated.
- **Critical Gap - File Operations:** `routers/files.py` currently sits at 32% coverage. While file handling represents only ∼4% of the backend codebase, it manages critical I/O. *Action Item:* Add 8-10 targeted

Table 10: Modules Requiring Additional Test Coverage

Module	**Coverage**	**Missed Lines**	**Identified Issue**
`services/visualization_service.py`	83%	244	Heatmap data transformation, NaN handling edge cases
`routers/visualizations.py`	74%	46	Multiple router error handling paths
`routers/ask.py`	79%	19	Streaming response logic untested
`routers/filters.py`	67%	15	Filtering validation logic not tested
`routers/files.py`	32%	57	Critical: File upload/-download logic largely untested

tests for file upload validation and stream response handling to secure this vector.

- **Moderate Gap - Visualization Edge Cases:** `visualization_service.py` requires 6-8 additional tests focusing heavily on boundary conditions, specifically unusual data combinations that result in NaN or infinite values during heatmap generation.

- **Frontend Blindspot:** The frontend application (TypeScript/React) currently contains 7,277 LOC but lacks an integrated testing library (e.g., Jest, Vitest). *Action Item:* Acknowledge this technical debt; future revisions must prioritize UI component testing.

Bibliography

References

Author Author. System requirements specification. https://github.com/..., 2019.

Carlo Ghezzi, Mehdi Jazayeri, and Dino Mandrioli. *Fundamentals of Software Engineering*. Prentice Hall, Upper Saddle River, NJ, USA, 2nd edition, 2003.

Daniel M. Hoffman and Paul A. Strooper. *Software Design, Automated Testing, and Maintenance: A Practical Approach*. International Thomson Computer Press, New York, NY, USA, 1995. URL http://citeseer.ist.psu.edu/428727.html.

David L. Parnas. On the criteria to be used in decomposing systems into modules. *Comm. ACM*, 15(2):1053–1058, December 1972.

David L. Parnas. Designing software for ease of extension and contraction. In *ICSE '78: Proceedings of the 3rd international conference on Software engineering*, pages 264–277, Piscataway, NJ, USA, 1978. IEEE Press. ISBN none.

David L. Parnas and P.C. Clements. A rational design process: How and why to fake it. *IEEE Transactions on Software Engineering*, 12(2):251–257, February 1986.

D.L. Parnas, P.C. Clement, and D. M. Weiss. The modular structure of complex systems. In *International Conference on Software Engineering*, pages 408–419, 1984.

James Robertson and Suzanne Robertson. *Volere Requirements Specification Template*. Atlantic Systems Guild Limited, 16 edition, 2012.

W. Spencer Smith. Systematic development of requirements documentation for general purpose scientific computing software. In *Proceedings of the 14th IEEE International Requirements Engineering Conference, RE 2006*, pages 209–218, Minneapolis / St. Paul, Minnesota, 2006. URL http://www.ifi.unizh.ch/req/events/RE06/.

W. Spencer Smith and Nirmitha Koothoor. A document-driven method for certifying scientific computing software for use in nuclear safety analysis. *Nuclear Engineering and Technology*, 48(2):404–418, April 2016. ISSN 1738-5733. doi: http://dx.doi.org/10.1016/j.net.2015.11.008. URL http://www.sciencedirect.com/science/article/pii/S1738573315002582.

W. Spencer Smith and Lei Lai. A new requirements template for scientific computing. In J. Ralyté, P. Ågerfalk, and N. Kraiem, editors, *Proceedings of the First International Workshop on Situational Requirements Engineering Processes – Methods, Techniques and Tools to Support Situation-Specific Requirements Engineering Processes, SREP'05*, pages 107–121, Paris, France, 2005. In conjunction with 13th IEEE International Requirements Engineering Conference.

W. Spencer Smith, Lei Lai, and Ridha Khedri. Requirements analysis for engineering computation: A systematic approach for improving software reliability. *Reliable Computing, Special Issue on Reliable Engineering Computation*, 13(1):83–107, February 2007.

W. Spencer Smith, John McCutchan, and Jacques Carette. Commonality analysis of families of physical models for use in scientific computing. In *Proceedings of the First International Workshop on Software Engineering for Computational Science and Engineering (SECSE 2008)*, Leipzig, Germany, May 2008. In conjunction with the 30th International Conference on Software Engineering (ICSE). URL http://www.cse.msstate.edu/~SECSE08/schedule.htm. 8 pp.

W. Spencer Smith, John McCutchan, and Jacques Carette. Commonality analysis for a family of material models. Technical Report CAS-17-01-SS, McMaster University, Department of Computing and Software, 2017.

Appendix — Usability Survey Questions

- Were the system's features and functionalities easy to find and intuitively placed?
- Did the system's functionalities respond quickly to your input?
- Were you aware of what the system was doing at all times and did you receive feedback after performing an action?
- Were the search and webstore functionalities easy to use?
- How well do the search filters satisfy your needs?
- Are the system's information display and data visualization clear and well organized?
- What were your least favourite parts of using the system to complete a workflow?
- On a scale of one to five, how easy was it to learn how to use the system (one meaning impossible and five meaning easy)?
- On a scale of one to ten, how was your experience using the system's UI (one meaning not user-friendly at all and ten meaning very user-friendly)?

Appendix — Reflection

The information in this section will be used to evaluate the team members on the graduate attribute of Reflection.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

   Something that contributed to how smooth the writing process went was the fact that we had a large portion of pre-existing information that allowed us to complete the VnV Report, largely coming from the VnV Plan. Not only did the VnV Plan provide us with a majority of the information we needed, but our consistent and lengthy meetings with our supervisors throughout the project allowed us to have a very clear understanding of the requirements, and finer-grained details for our implementation, and from the previously existing implmentations made by them and the previous capstone team.

2. What pain points did you experience during this deliverable, and how did you resolve them?

   Frankly, as students in our final month of university, the finish line is very much in sight and thus it is becoming difficult to stay locked in and motivated to complete assignments to the best of our abilities. Especially considering the fact that writing a verification and validation report is not the most interesting thing to be doing (when compared to our other, more technical classes). So, this specific deliverable was subject to quite a bit of procrastination, most of it being completed

the weekend before the due date (it is currently 5:11 p.m. on Monday). Hopefully, this doesn't result in too much of a decrease in quality but we are all at the point that we won't be overly bothered if that is the case.

Another pain point was that the VnV Plan was not completed yet. The unit testing section was not supposed to be completed when we first wrote the plan and we never got around to it after drafting our system design. So, we essentially had 4 additional sections that needed to be completed than we initially thought, which unexpectedly added to our workload and caused some issues with task delegation and timely completion.

3. Which parts of this document stemmed from speaking to your client(s) or a proxy (e.g. your peers)? Which ones were not, and why?

   As software engineering students, we are not domain experts in the field of neuroscience, nor animal biology or obsessive compulsive disorder. In order to adequately validate our system, we relied heavily on the domain expertise of our clients to ensure alignment with the requirements and expectations of the project. For example, to validate the system's natural language capabilities, we worked with our supervisor to establish a mapping between numerous sample queries and expected results. Frontloading these kinds of validation activities allowed us to independently verify the system's capabilities in a way that we would not have been able to do on our own.

   System components that were independent of domain expertise, such as the API behaviour under various error conditions, or connectivity with external systems including the LLM and the FRDR database were major portions of the document that could be completed without client input. In these cases, the team was equipped to discern the expections of the product without external validation. In these cases where the team was able to self-validate certain subsystems, we still ensured indirect supervisor validation, typically via an interactive demo of the corresponding feature tied to the subsystem.

4. In what ways was the Verification and Validation (VnV) Plan different from the activities that were actually conducted for VnV? If there were differences, what changes required the modification in the plan? Why

did these changes occur? Would you be able to anticipate these changes in future projects? If there weren't any differences, how was your team able to clearly predict a feasible amount of effort and the right tasks needed to build the evidence that demonstrates the required quality? (It is expected that most teams will have had to deviate from their original VnV Plan.)

The Verification and Validation activities conducted during this project deviated from the original VnV Plan in one primary structural way: the granularity of unit tests. Each unit test outlined in the VnV Plan was written at a higher level of abstraction, generally corresponding to one core requirement or function that might map to several requirements. However, during implementation, these higher-level unit tests were decomposed into more granular, function-level test cases. As a result, a single unit test described in the VnV Plan is often represented by multiple concrete tests in the PyTest suite located in the `src/ocd-rat-backend/tests` directory.

This change occurred naturally as part of the development process. When translating high-level test intentions into executable test code, it became clear that individual functions required isolated verification, leading to a finer-grained test structure than originally anticipated. Revamping the VnV Plan to reflect this increased granularity was deemed impractical — it would have been highly tedious both to write and to evaluate — whereas the existing higher-level overview provides a more comprehensive view of the motivation behind the tests. Therefore, the VnV Plan was intentionally left as written.

Beyond the structural granularity difference, testing also surfaced several functional issues not fully anticipated in the plan, including: concurrent download failures (resolved via background tasks with hashed job IDs), SQL injection risks identified through CodeQL static analysis (resolved by wrapping user inputs), and insufficient error handling in the download and inventory filter endpoints (resolved with more robust `try/catch` blocks). Additionally, several anticipated issues were uncovered but remain unresolved, such as slow download performance for large datasets, malformed line chart rendering due to missing date values, path traversal vulnerabilities in download logic, inaccurate session-to-file-count mapping, and brittle keyword-based chat mode detection.

In future projects, some of these deviations could be anticipated. The granularity gap between plan-level and implementation-level tests is a common and foreseeable consequence of translating requirements into code. Planning for test decomposition explicitly — or writing the VnV Plan after an initial implementation pass — could reduce this disconnect. Security and concurrency issues, however, are harder to predict without first engaging in implementation, and thus may inherently require an iterative approach to VnV planning.